

5월20일 데이터베이스 수업자료

연습문제 19(128page)

[여행사 데이터베이스]다음은 항공사 및 여행사 관련 데이터베이스에 대한 릴레이션이다.
밑줄친 속성은 기본키이며, 이름이 같은 속성은 외래키로 사용된다.

Passenger(<u>pid</u> , pname, pgender, pcity)	//승객정보
Agency(<u>aid</u> , aname, acity)	//여행사 정보
Flight(<u>fid</u> , fdate, time, src, dest)	//항공편 정보
Booking(<u>pid</u> , <u>aid</u> , <u>fid</u> , fdate)	//예약정보

```
-- =====
-- 테이블 생성
-- =====

-- 승객 테이블
CREATE TABLE Passenger (
    pid    NUMBER          PRIMARY KEY,
    pname  VARCHAR2(100)   NOT NULL,
    pgender VARCHAR2(5),
    pcity  VARCHAR2(100)
);

-- =====
-- 테이블 생성
-- =====

-- 승객 테이블
CREATE TABLE Passenger (
    pid    NUMBER          PRIMARY KEY,
    pname  VARCHAR2(100)   NOT NULL,
    pgender VARCHAR2(5),
    pcity  VARCHAR2(100)
);

-- 여행사 테이블
CREATE TABLE Agency (
    aid    NUMBER          PRIMARY KEY,
    aname  VARCHAR2(100)   NOT NULL,
```

```

        acity    VARCHAR2(100)
    );

-- 항공편 테이블
CREATE TABLE Flight (
    fid        NUMBER          PRIMARY KEY,
    fdate      DATE            NOT NULL,
    time       VARCHAR2(10),
    src        VARCHAR2(100),
    dest       VARCHAR2(100)
);

-- 예약 테이블
CREATE TABLE Booking (
    pid        NUMBER,
    aid        NUMBER,
    fid        NUMBER,
    fdate      DATE,
    CONSTRAINT pk_booking    PRIMARY KEY (pid, aid, fid),
    CONSTRAINT fk_book_pass  FOREIGN KEY (pid) REFERENCES Passenger(pid),
    CONSTRAINT fk_book_agen  FOREIGN KEY (aid) REFERENCES Agency(aid),
    CONSTRAINT fk_book_flg   FOREIGN KEY (fid) REFERENCES Flight(fid)
);

-- =====
-- 예시 데이터 삽입
-- =====

-- 승객 데이터 (승객번호 100번 포함)
INSERT INTO Passenger VALUES (100, '김철수', '남', '서울시 강남구');
INSERT INTO Passenger VALUES (101, '이영희', '여', '서울시 강동구');
INSERT INTO Passenger VALUES (102, '박민준', '남', '서울시 도봉구');
INSERT INTO Passenger VALUES (103, '최수연', '여', '서울시 마포구');
INSERT INTO Passenger VALUES (104, '홍길동', '남', '서울시 송파구');

-- 여행사 데이터
INSERT INTO Agency VALUES (1, '마당여행사', '서울시 강남구');
INSERT INTO Agency VALUES (2, '하늘여행사', '서울시 강동구');
INSERT INTO Agency VALUES (3, '바람여행사', '서울시 도봉구');

```

```

-- 항공편 데이터 (항공편 100번: 김포→제주 포함)
INSERT INTO Flight VALUES (100, TO_DATE('2025-01-15', 'YYYY-MM-DD'), '09:00',
'김포', '제주');
INSERT INTO Flight VALUES (101, TO_DATE('2025-01-20', 'YYYY-MM-DD'), '11:00',
'김포', '제주');
INSERT INTO Flight VALUES (102, TO_DATE('2025-01-25', 'YYYY-MM-DD'), '14:00',
'김포', '부산');
INSERT INTO Flight VALUES (103, TO_DATE('2024-12-31', 'YYYY-MM-DD'), '16:00',
'김포', '제주'); -- 2025년 이전
INSERT INTO Flight VALUES (104, TO_DATE('2025-02-01', 'YYYY-MM-DD'), '18:00',
'부산', '제주');

-- 예약 데이터 (승객 100번이 2025년 1월 1일 이후 탑승 포함)
-- 100번 승객 예약 (2025년 이후)
INSERT INTO Booking VALUES (100, 1, 100, TO_DATE('2025-01-15',
'YYYY-MM-DD')); -- ✔ 김포→제주 100번 항공편
INSERT INTO Booking VALUES (100, 2, 101, TO_DATE('2025-01-20',
'YYYY-MM-DD')); -- ✔ 김포→제주 101번 항공편
INSERT INTO Booking VALUES (100, 1, 104, TO_DATE('2025-02-01',
'YYYY-MM-DD')); -- ✔ 부산→제주

-- 100번 승객 예약 (2025년 이전 - 비교용)
INSERT INTO Booking VALUES (100, 3, 103, TO_DATE('2024-12-31',
'YYYY-MM-DD')); -- ✖ 2025년 이전

-- 다른 승객 예약
INSERT INTO Booking VALUES (101, 1, 100, TO_DATE('2025-01-15',
'YYYY-MM-DD'));
INSERT INTO Booking VALUES (102, 2, 101, TO_DATE('2025-01-20',
'YYYY-MM-DD'));
INSERT INTO Booking VALUES (103, 3, 102, TO_DATE('2025-01-25',
'YYYY-MM-DD'));
INSERT INTO Booking VALUES (104, 1, 103, TO_DATE('2024-12-31',
'YYYY-MM-DD'));

COMMIT;

-- =====

```

```

-- 검증 쿼리
-- =====

-- 1) 항공편 100번인 승객이 2025년 1월 1일 이후 탑승한 데이터
SELECT p.pid, p.pname, f.fid, f.src, f.dest, b.fdate
FROM Passenger p
JOIN Booking b ON p.pid = b.pid
JOIN Flight f ON b.fid = f.fid
WHERE f.fid = 100
AND b.fdate > TO_DATE('2025-01-01', 'YYYY-MM-DD');

-- 2) 승객 100번이 2025년 1월 1일 이후 탑승한 데이터
SELECT p.pid, p.pname, f.fid, f.src, f.dest, b.fdate
FROM Passenger p
JOIN Booking b ON p.pid = b.pid
JOIN Flight f ON b.fid = f.fid
WHERE p.pid = 100
AND b.fdate > TO_DATE('2025-01-01', 'YYYY-MM-DD');
...

-

```

1)도착지가 제주인 항공편에 대한 정보를 보이시오

```

SELECT *
FROM Flight
WHERE dest = '제주';

```

2)출발지가 김포(src)이고 도착지가 제주(dest)인 항공편에 대한 정보를 보이시오

```

SELECT *
FROM Flight
WHERE src = '김포'
AND dest = '제주';

```

3)고객번호가 100번인 승객이 2025년1월 1일 이후에 탑승한 비행기 번호(fid)를 보이시오

```

SELECT DISTINCT b.fid
FROM Booking b
WHERE b.pid = 100
AND b.fdate > TO_DATE('2025-01-01', 'YYYY-MM-DD');

```

4)예약을 한 적이 있는 고객의 이름(pname)을 보이시오

```

SELECT DISTINCT p.pname
FROM Passenger p
WHERE EXISTS (
    SELECT 1
    FROM Booking b
    WHERE b.pid = p.pid );

```

5)예약을 한 적이 없는 고객의 이름(pname)을 보이시오

```

SELECT DISTINCT p.pname
FROM Passenger p
WHERE NOT EXISTS (
    SELECT 1
    FROM Booking b
    WHERE b.pid = p.pid );

```

6)고객번호가 100번인 승객이 거주하는 도시(pcity)와 같은 도시에 위치한 여행사(aname)의 이름을 보이시오

```

SELECT a.aname
FROM Agency a
WHERE a.acity = (
    SELECT p.pcity
    FROM Passenger p
    WHERE p.pid = 100 );

```

7)2025년 1월 1일부터 1월 30일 사이에 출발시각이 16:00이후인 항공편 정보를 보이시오

```

SELECT *
FROM Flight
WHERE fdate BETWEEN TO_DATE('2025-01-01', 'YYYY-MM-DD')
    AND TO_DATE('2025-01-30', 'YYYY-MM-DD')
    AND time >= '16:00';

```

보충설명

Oracle에서 time이 VARCHAR2로 저장된 경우와 DATE로 저장된 경우에 따라 다르게 처리해야 함

```

sql-- time이 VARCHAR2('HH:MM')인 경우
AND time >= '16:00'
-- time이 DATE 타입인 경우
AND TO_CHAR(time, 'HH24:MI') >= '16:00'

```

8)고객번호가 100번인 승객이 한 번도 예약하지 않은 여행사의 이름(aname)을 보이시오

```

SELECT a.aname

```

```
FROM Agency a
WHERE NOT EXISTS (
    SELECT 1
    FROM Booking b
    WHERE b.aid = a.aid
    AND b.pid = 100 );
```

보충설명 (서브쿼리의 select 1-조건에 맞는 행이 존재하는지만 확인)

```
① SELECT 1 사용
WHERE NOT EXISTS (
    SELECT 1
    FROM Booking b
    WHERE b.aid = a.aid AND b.pid = 100
);
-- ② SELECT * 사용
WHERE NOT EXISTS (
    SELECT *
    FROM Booking b
    WHERE b.aid = a.aid AND b.pid = 100
);
-- ③ SELECT b.aid 사용
WHERE NOT EXISTS (
    SELECT b.aid
    FROM Booking b
    WHERE b.aid = a.aid AND b.pid = 100
);
```

9)마당여행사(aname)를 통해 예약한 남자 승객(pgender)의 정보를 보이시오

```
SELECT DISTINCT p.*
FROM Passenger p
JOIN Booking b ON p.pid = b.pid
JOIN Agency a ON b.aid = a.aid
WHERE a.aname = '마당여행사'
AND p.pgender = '남';
```

[단순질의]

1. Passenger 테이블에서 모든 승객의 pid, pname, pcity를 조회하시오.
(풀이)
2. Passenger 테이블에서 pgender가 '남'인 승객의 pname과 pcity를 조회하시오.
(풀이)
3. Flight 테이블에서 출발지(src)가 '김포'인 항공편의 fid, fdate, dest를 조회하시오.
(풀이)
4. Booking 테이블에서 pid별 예약 건수를 조회하시오.
(풀이)
5. Agency 테이블에서 모든 여행사의 aid, aname, acity를 조회하시오.
(풀이)
6. Flight 테이블에서 목적지(dest)가 '제주'인 항공편 수를 조회하시오.
(풀이)
7. Booking 테이블에서 fdate가 '2025-01-01' 이후인 예약의 pid, aid, fid를 조회하시오.
(풀이)
8. Booking 테이블에서 aid별 예약 건수를 내림차순으로 조회하시오.
(풀이)
9. Flight 테이블에서 fdate가 가장 최근인 항공편의 fid와 src, dest를 조회하시오.
(풀이)
10. Passenger 테이블에서 pcity가 '서울시 강남구' 또는 '서울시 강동구'인 승객의 pname을 조회하시오.
(풀이)

[조인질의]

11. Passenger와 Booking 테이블을 조인하여 승객 이름(pname)과 예약한 fid를 조회하시오.
(풀이)
12. Agency와 Booking 테이블을 조인하여 여행사 이름(aname)과 예약 건수를 조회하시오.
(풀이)
13. Passenger, Booking, Flight 테이블을 조인하여 승객 이름, 출발지(src), 목적지(dest)를 조회하시오.
(풀이)
14. Passenger, Booking, Agency 테이블을 조인하여 승객 이름과 이용한 여행사 이름을 조회하시오. (중복 제거)
(풀이)
15. Passenger와 Booking 테이블을 LEFT JOIN하여 예약이 한 건도 없는 승객의 pname을 조회하시오.
(풀이)

16. Flight와 Booking 테이블을 조인하여 2025년 이후 항공편의 목적지(dest)별 예약 건수를 조회하시오.

(풀이)

17. Passenger, Booking, Flight 테이블을 조인하여 '제주'행 항공편을 예약한 승객 이름을 중복 없이 조회하시오.

(풀이)

18. Passenger, Booking, Agency 테이블을 조인하여 '마당여행사'를 통해 예약한 승객의 이름을 조회하시오. (중복 제거)

(풀이)

19. Passenger, Booking, Flight, Agency 4개 테이블을 조인하여 승객 이름, 여행사 이름, 출발지, 목적지를 조회하시오. (2025년 예약만)

(풀이)

20. Flight와 Booking 테이블을 LEFT JOIN하여 예약이 없는 항공편의 fid, src, dest를 조회하시오.

(풀이)

[부속질의]

21. Passenger 테이블에서 Booking 테이블에 예약 기록이 있는 승객의 pname을 조회하시오. (IN 사용)

(풀이)

22. Passenger 테이블에서 2025년 이후 항공편을 예약한 승객의 pname을 조회하시오.

(풀이)

23. Agency 테이블에서 예약 건수가 3건 이상인 여행사의 aname을 조회하시오.

(풀이)

24. Flight 테이블에서 Booking 테이블에 예약된 적 없는 항공편의 fid, src, dest를 조회하시오. (NOT IN 사용)

(풀이)

25. Passenger 테이블에서 예약 건수가 가장 많은 승객의 pname을 조회하시오.

(풀이)

26. Passenger 테이블에서 '부산'행 항공편을 예약한 승객의 pname을 조회하시오.

(풀이)

27. Agency 테이블에서 예약이 한 건도 없는 여행사의 aname을 조회하시오. (NOT IN 사용)

(풀이)

28. Passenger 테이블에서 '마당여행사(aid=1)'를 통해 예약한 승객의 pname을 조회하시오.

(풀이)

29. Booking 테이블에서 예약 건수가 전체 승객 평균 예약 건수보다 많은 pid를 조회하시오.

(풀이)

30. Passenger 테이블에서 2024년에 예약한 기록이 있는 승객의 pname을 조회하시오.

(풀이)

[상관부속질의]

31. Passenger 테이블에서 Booking에 본인 pid로 예약 기록이 존재하는 승객의 pname을 조회하시오. (EXISTS 사용)

(풀이)

32. Passenger 테이블에서 본인의 예약 건수가 2건 이상인 승객의 pname을 조회하시오. (상관 서브쿼리 사용)

(풀이)

33. Flight 테이블에서 Booking에 예약 기록이 전혀 없는 항공편의 fid를 조회하시오. (NOT EXISTS 사용)

(풀이)

34. Passenger 테이블에서 2025년 이후 항공편을 예약한 기록이 존재하는 승객의 pname을 조회하시오. (EXISTS 사용)

(풀이)

35. Passenger 테이블에서 서로 다른 여행사(aid)를 2곳 이상 이용한 승객의 pname을 조회하시오.

(풀이)

36. Agency 테이블에서 Booking에 예약이 존재하지 않는 여행사의 aname을 조회하시오. (NOT EXISTS 사용)

(풀이)

37. Passenger 테이블에서 본인이 예약한 항공편 중 '제주'행이 존재하는 승객의 pname을 조회하시오.

(풀이)

38. Passenger 테이블에서 2024년에만 예약하고 2025년에는 예약이 없는 승객의 pname을 조회하시오.

(풀이)

39. Flight 테이블에서 같은 목적지(dest) 항공편 중 예약 건수가 가장 많은 항공편의 fid를 조회하시오.

(풀이)

40. Passenger 테이블에서 본인이 예약한 모든 항공편의 출발지가 '김포'인 승객의 pname을 조회하시오. (NOT EXISTS 활용)

(풀이)

연습문제 20(128page)

[기업프로젝트 데이터베이스] 다음은 기업의 프로젝트 관리와 관련된 데이터베이스 릴레이션이다.

Employee는 사원, Department는 부서, Project는 프로젝트, Works는 사원이 프로젝트에 참여한 내역을 나타낸다. 한 사람은 여러 프로젝트에 참여할 수 있으며, 하나의 프로젝트에도 여러 사원이 함께 일할 수 있다. hours_worked 속성은 각 사원이 특정 프로젝트에서 일한 시간을 나타낸다. 또한, Department속성에는 사원의 사번(empno)이 저장된다고 가정한다.

Employee(empno, name, phoneno, address, sex, position, deptno)

Department(deptno, deptname, manager)

Project(projno, projname, deptno)

Works(empno,projno, hours_worked)

1)각 릴레이션의 기본키를 정하십시오

```
-- 4번 vs 5번
-- 예약 있는 고객 → EXISTS
WHERE EXISTS (SELECT 1 FROM Booking b WHERE b.pid = p.pid)

-- 예약 없는 고객 → NOT EXISTS
WHERE NOT EXISTS (SELECT 1 FROM Booking b WHERE b.pid = p.pid)

-- 6번 스칼라 서브쿼리
-- 100번 승객 도시를 먼저 구한 후 여행사 도시와 비교
WHERE a.acity = (SELECT pcity FROM Passenger WHERE pid = 100)

-- 8번 특정 고객 미예약 여행사
-- 100번 승객이 예약한 여행사를 제외한 나머지
WHERE NOT EXISTS (SELECT 1 FROM Booking b
                    WHERE b.aid = a.aid AND b.pid = 100)
```

2)릴레이션 간의 관계를 고려하여 외래기를 찾아보시오

1) 기본키

릴레이션	기본키
Employee	empno
Department	deptno
Project	projno
Works	empno + projno (복합키)

2) 외래키

릴레이션	외래키	참조 테이블
Employee	deptno	Department(deptno)
Department	manager	Employee(empno)
Project	deptno	Department(deptno)
Works	empno	Employee(empno)
Works	projno	Project(projno)

-- 테이블 생성

```
CREATE TABLE Employee (
```

```
    empno    NUMBER          PRIMARY KEY,  
    name     VARCHAR2(100)   NOT NULL,  
    phoneno  VARCHAR2(20),  
    address  VARCHAR2(200),  
    sex      VARCHAR2(5),  
    position VARCHAR2(50),  
    deptno   NUMBER
```

```
);
```

```
CREATE TABLE Department (
```

```
    deptno   NUMBER          PRIMARY KEY,  
    deptname VARCHAR2(100)   NOT NULL,  
    manager  NUMBER
```

```
);
```

```
CREATE TABLE Project (
```

```
    projno   NUMBER          PRIMARY KEY,  
    projname VARCHAR2(100)   NOT NULL,  
    deptno   NUMBER
```

```
);
```

```
CREATE TABLE Works (
```

```
    empno      NUMBER,  
    projno     NUMBER,  
    hours_worked NUMBER,  
    CONSTRAINT pk_works PRIMARY KEY (empno, projno),  
    CONSTRAINT fk_works_emp FOREIGN KEY (empno) REFERENCES Employee(empno),  
    CONSTRAINT fk_works_proj FOREIGN KEY (projno) REFERENCES Project(projno)
```

```
);
```

-- 외래키 추가

```
ALTER TABLE Employee
ADD CONSTRAINT fk_emp_dept FOREIGN KEY (deptno) REFERENCES Department(deptno);
```

```
ALTER TABLE Department
ADD CONSTRAINT fk_dept_mgr FOREIGN KEY (manager) REFERENCES Employee(empno);
```

```
ALTER TABLE Project
ADD CONSTRAINT fk_proj_dept FOREIGN KEY (deptno) REFERENCES Department(deptno);
```

```
-- =====
```

```
-- 예시 데이터 삽입
```

```
-- =====
```

```
-- Department (FK 순환 때문에 manager 나중에 UPDATE)
```

```
INSERT INTO Department VALUES (1, 'IT부서', NULL);
```

```
INSERT INTO Department VALUES (2, '영업부서', NULL);
```

```
INSERT INTO Department VALUES (3, '기획부서', NULL);
```

```
-- Employee
```

```
INSERT INTO Employee VALUES (101, '김철수', '010-1111-1111', '서울시 강남구', '남', '부장', 1);
```

```
INSERT INTO Employee VALUES (102, '이영희', '010-2222-2222', '서울시 강동구', '여', '과장', 1);
```

```
INSERT INTO Employee VALUES (103, '박민준', '010-3333-3333', '서울시 도봉구', '남', '대리', 2);
```

```
INSERT INTO Employee VALUES (104, '최수연', '010-4444-4444', '서울시 마포구', '여', '사원', 2);
```

```
INSERT INTO Employee VALUES (105, '정하늘', '010-5555-5555', '서울시 송파구', '남', '과장', 3);
```

```
INSERT INTO Employee VALUES (106, '한지민', '010-6666-6666', '서울시 용산구', '여', '사원', 1);
```

```
-- Department manager 업데이트
```

```
UPDATE Department SET manager = 101 WHERE deptno = 1;
```

```
UPDATE Department SET manager = 103 WHERE deptno = 2;
```

```
UPDATE Department SET manager = 105 WHERE deptno = 3;
```

```
-- Project
```

```
INSERT INTO Project VALUES (1, '미래', 1);
```

```
INSERT INTO Project VALUES (2, '스마트시티', 2);
```

```
INSERT INTO Project VALUES (3, '디지털전환', 1);
```

```
-- Works
```

```
INSERT INTO Works VALUES (101, 1, 40);
```

```
INSERT INTO Works VALUES (102, 1, 35);
```

```
INSERT INTO Works VALUES (103, 2, 50);
```

```
INSERT INTO Works VALUES (104, 2, 30);
```

```
INSERT INTO Works VALUES (105, 3, 45);
```

```
INSERT INTO Works VALUES (106, 1, 20);
```

```
INSERT INTO Works VALUES (101, 3, 25);
```

COMMIT;

3) 모든 직원의 이름을 보이시오

```
SELECT name FROM Employee;
```

결과] 김철수, 이영희, 박민준, 최수연, 정하늘, 한지민

4) 성별이 여자인 직원의 이름을 보이시오

```
SELECT name FROM Employee WHERE sex = '여';
```

결과] 이영희, 최수연, 한지민

5) 부서장(manager)의 이름과 주소를 보이시오

```
SELECT e.name, e.address FROM Employee e
      JOIN Department d ON e.empno = d.manager;
```

결과] 이름 주소

김철수	서울시 강남구	← IT부서 부서장
박민준	서울시 도봉구	← 영업부서 부서장
정하늘	서울시 송파구	← 기획부서 부서장

6) IT부서에서 근무하는 직원의 이름과 주소를 보이시오

```
SELECT e.name, e.address FROM Employee e
      JOIN Department d ON e.deptno = d.deptno
      WHERE d.deptname = 'IT부서';
```

결과] 이름 주소

김철수	서울시 강남구
이영희	서울시 강동구
한지민	서울시 용산구

7) '미래'라는 이름의 프로젝트에서 일하는 직원의 이름을 보이시오

```
SELECT DISTINCT e.name FROM Employee e
      JOIN Works w   ON e.empno = w.empno
      JOIN Project p  ON w.projno = p.projno
      WHERE p.projname = '미래';
```

결과] 이름

김철수
이영희
한지민